

PIM – Projeto Integrado Multidisciplinar

UNIP - Universidade Paulista

Projeto Integrado Multidisciplinar III

**Desenvolvimento de uma plataforma integrada para
gestão de pesquisa clínica e produção farmacêutica**

Santana de Parnaíba, SP

2025

Projeto Integrado Multidisciplinar III

Desenvolvimento de uma plataforma integrada para gestão de pesquisa clínica e produção farmacêutica

Projeto Integrado Multidisciplinar entregue à UNIP

Para a avaliação no curso Superior em

Gestão da Tecnologia da Informação

Santana de Parnaíba, SP

2025

Curso Superior em Gestão da Tecnologia da Informação

UNIP – UNIVERSIDADE PAULISTA

Autores	R.A
Yasmine Lima Santana	2526592
Luan José Marques da Costa	2548014
Geovanna Félix Monteiro	2521699
Luan Lopes Bezerra	2503460
João Pedro Carneiro da Silva	2540975
Vitória Freitas	2523279
Sofia de Castro Ongaro	2528018

Santana de Parnaíba, SP

2025

Resumo

Este trabalho apresenta o desenvolvimento de uma plataforma digital voltada à gestão de pesquisas clínicas e processos farmacêuticos, com foco na segurança dos dados, eficiência dos sistemas e sustentabilidade ambiental.

Na primeira parte, foi utilizado o método **Scrum**, uma forma ágil de gerenciar projetos que divide o trabalho em etapas menores chamadas sprints. Essa metodologia permite que o sistema seja construído e testado aos poucos, facilitando correções e melhorias. Também foram aplicadas métricas como Velocity e Burndown Chart, que ajudam a acompanhar o progresso da equipe e a garantir entregas de qualidade.

Na segunda parte, o projeto foi desenvolvido em **Python**, utilizando estruturas de dados e aprendizado de máquina para organizar informações sobre pacientes e medicamentos. Foram implementadas ferramentas como **árvores AVL** e **grafos**, que tornam as buscas e análises mais rápidas e seguras. O código também passou por testes e verificações para garantir que os resultados fossem confiáveis e precisos.

Por fim, o trabalho aborda a importância da **educação ambiental** dentro da área farmacêutica. A plataforma propõe um módulo educativo que ensina o descarte correto de medicamentos, evitando a contaminação do solo e da água. Essa integração entre tecnologia e consciência ambiental reforça a responsabilidade social e a importância de unir inovação com cuidado ao meio ambiente.

De forma geral, o projeto busca contribuir para um futuro mais sustentável, onde a tecnologia, a saúde e a preservação ambiental caminhem juntas para gerar impacto positivo na sociedade.

Palavras-chave: Engenharia de Software. Python. Sustentabilidade. Educação Ambiental. Descarte de Medicamentos.

Abstract

This paper presents the development of a digital platform focused on managing clinical research and pharmaceutical production processes, emphasizing data security, system efficiency, and environmental sustainability.

In the first stage, the **Scrum** method was applied — an agile project management framework that divides work into smaller cycles called sprints. This approach allows the system to be built and tested gradually, making it easier to implement corrections and improvements. Metrics such as Velocity and Burndown Chart were also used to monitor team progress and ensure quality deliveries.

In the second stage, the project was developed in **Python**, using data structures and machine learning techniques to organize information about patients and medications. Tools such as **AVL trees** and **graphs** were implemented to make data search and analysis faster and more secure. The code also underwent testing and validation to guarantee reliable and accurate results.

Finally, the study highlights the importance of **environmental education** in the pharmaceutical field. The platform includes an educational module that guides users on the proper disposal of medicines, helping to prevent soil and water contamination. This integration of technology and environmental awareness reinforces social responsibility and demonstrates how innovation and sustainability can work together.

Overall, this project aims to contribute to a more sustainable future, where technology, health, and environmental preservation coexist to generate positive social impact.

Keywords: Software Engineering. Python. Sustainability. Environmental Education. Drug Disposal.

Sumário

Introdução	7
1. Engenharia de Software Ágil	8
1.1 Planejamento do Projeto	8
1.2 Execução Ágil	9
1.3 Sprints	9
1.4 Métricas: Velocity e Burndown Chart	10
1.5 Entregáveis e Controle de Qualidade	11
1.6 Continuidade do Projeto	12
2. Algoritmos e Estruturas de Dados em Python	13
2.1 Explicação geral do Código Fonte do Projeto	13
6. Educação Ambiental	42
3.1 Introdução	42
3.2 Justificativa	43
3.3 Objetivos	43
3.4 Fundamentação Teórica	43
3.5 Metodologia	44
3.6. Atividade de Extensão	44
3.7 Resultados Esperados	46
Conclusão	47
Bibliografias	48

Introdução

O avanço da tecnologia tem transformado a forma como cuidamos da saúde e do meio ambiente. Cada vez mais, a área farmacêutica depende de sistemas digitais para organizar dados, acompanhar pesquisas e garantir que todos os processos sejam seguros e eficientes.

Pensando nisso, este trabalho propõe o desenvolvimento de uma plataforma que une **gestão de pesquisas clínicas, controle farmacêutico e educação ambiental**. A ideia é facilitar o acesso às informações, melhorar o acompanhamento dos resultados e, ao mesmo tempo, conscientizar sobre o **descarte correto de medicamentos**, evitando danos ao meio ambiente e à saúde pública.

Além de contribuir para o avanço tecnológico, a proposta também reforça a importância da **responsabilidade social e ambiental**, mostrando que inovação e sustentabilidade podem caminhar juntas para um futuro mais equilibrado e seguro.

1. ENGENHARIA DE SOFTWARE ÁGIL

1.1 Planejamento do Projeto

Para o planejamento do projeto, escolhemos que o processo de desenvolvimento será feito com Scrum, utilizando o VS Code (Visual Studio) para codificação, com revisões de código e teste unitários com a ferramenta unittest.

O Scrum é uma metodologia ágil que consiste no gerenciamento de projetos complexos focado em dividir o projeto em ciclos pequenos onde a equipe concentra em entregar partes do trabalho de forma incremental e iterativo tornando o processo mais simples.

E antes de inicializar o planejamento de um projeto uma série de levantamentos de requisitos e necessidades do usuário deve ser considerado, criando assim um Caso de uso onde será priorizada a história do usuário, que nada mais é a forma que o usuário vai descrever como ele deseja o seu produto e suas necessidades.

Exemplo: “O usuário solicitou que em seu produto, ele sendo o gerente, tivesse acesso a dados da gestão, pesquisas laboratoriais farmacêuticas e descarte de resíduos contaminados, para que cada processo esteja seguindo normas, padrões e leis., também solicitou que seu produto tenha uma segurança melhor para não haver vazamento de dados e roubo de senhas, solicitou também que o sistema tenha uma resposta rápida para otimizar seu tempo”.

E o Product Owner irá traduzir para a equipe de programação de forma técnica, separando assim requisitos funcionais e não funcionais que o produto receberá, alinhando o cliente com o programador para que haja coerência entre o que foi solicitado e o que foi entregue e se a necessidade de alteração como adicionando, removendo ou corrigindo algo em seu produto e cada um desses partes se torna um item no Product Backlog.

O **Backlog** é um termo usado em qualquer área de gestão de trabalho, seja era Ti, Marketing suporte etc, e ela se refere a uma lista de demandas, pedidos ou tarefas que precisam ser realizadas. E o responsável por isso tudo é o Product Owner. O Product Owner é o responsável por maximizar o valor do produto e é ponto central

para todas as decisões sobre o produto. Ele é o elo vital entre o mundo dos negócios (stakeholders, mercado, clientes) e o mundo técnico (equipe de desenvolvimento), garantindo que o time construa o produto certo, na ordem certa, para maximizar o valor entregue.

1.2 Execução Ágil

No contexto de desenvolvimento de uma plataforma sobre a gestão de pesquisas clínicas e processos farmacêuticos, exige constante atenção a requisitos regulatórios, dados sensíveis e exigências ambientais, como discutido anteriormente. Destarte, o uso de execuções ágeis, como o Scrum, um framework, ajuda a organizar projetos de forma ágil, garantindo inspeções frequentes e por ser adaptável nas exigências da OMS (Organização Mundial de Saúde). Contribuindo para uma constante agilidade e evolução em conformidade com as normas FDA 21 CFR Part 11 (Regras de registros e assinaturas eletrônicas na Indústria Farmacêutica e de Ciências biológicas), GxP (Qualidade e segurança do produto, principalmente nas indústrias farmacêuticas e alimentícias) e ISO 14001 (Norma internacional com requisitos para um Sistema de Gestão Ambiental eficaz, visando a sustentabilidade e redução de impactos ambientais).

“O desenvolvimento ágil não é apenas uma metodologia, mas uma mentalidade focada em adaptação contínua e colaboração para entregar valor ao cliente.”

(PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8ª ed. McGraw Hill, 2016.)

1.3 Sprints

A Sprint, é um curto período e fixo em que uma equipe de scrum trabalha com o objetivo de concluir uma quantidade definida de trabalho, sendo incrementos funcionais para o sistema. Tendo em vista o cenário farmacêutico, as Sprints teriam como funcionalidade à implementação de um módulo essencial. Como por exemplo:

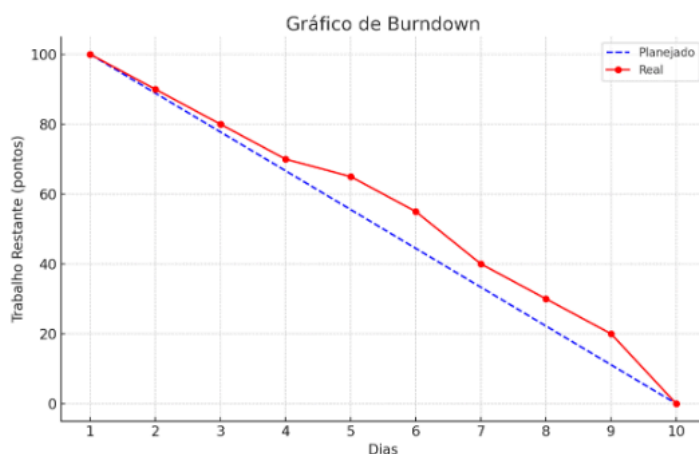
- Sprint 1 - Autentificações multifatoriais, em conformidade com a FDA 21 CFR Part 11, citada anteriormente.
- Sprint 2 - Implementação de infográficos, para quantificar o uso de energia e redução de impactos ambientais.

- Sprint 3 - Averiguar para que não haja contas com os mesmos dados pessoais, ou alteração dos mesmos.

Garantindo uma funcionalidade, segurança e confiabilidade maior ao programa, passando mais credibilidade tanto para os profissionais quanto para os pacientes. É importante enfatizar a necessidade de cada função ser dividida em pequenos blocos, o exemplo acima poderia ser subdividido caso houvesse a necessidade de uma redução de acoplamento.

1.4 Métricas: Velocity e Burndown Chart

Velocity e Burndown Chart, são métricas visuais no desenvolvimento e execução ágil que quando completas mostram o progresso da equipe em formato de gráficos quantitativos, para analisar, metas, prazos e entre outros fatores. Exemplo:



(créditos: PromoveSoluções)

O Velocity, tem como desempenho principal medir a quantidade de funções da equipe concluídas por Sprint, dessa forma a Velocity consegue estimar o desempenho da equipe. No cenário farmacêutico, esta função pode ser utilizada, quanto a metas de vendas de determinados produtos, aplicações, resultado de campanhas de conscientização de vacinação, funcionalidades regulatórias, entre outros.

Já o burndown chart, é um gráfico que aponta todo o trabalho restante com relação ao tempo disponível. Auxiliando a equipe para saber seu desenvolvimento contínuo,

e para monitoramento de prazos. O uso dessas métricas ajuda na previsibilidade do desenvolvimento da equipe e aponta melhoras.

1.5 Entregáveis e Controle de Qualidade

Entregáveis do Projeto

Os entregáveis representam os produtos, artefatos e resultados gerados durante o ciclo de vida do projeto de desenvolvimento do sistema farmacêutico. Eles asseguram que cada etapa seja documentada e auditável, de acordo com normas de qualidade e regulamentação internacional.

Principais entregáveis:

- Documento de Requisitos (Backlog Detalhado): contém as histórias do usuário, os requisitos funcionais e não funcionais, priorizados conforme as necessidades do cliente e os padrões regulatórios (FDA 21 CFR Part 11, GxP e ISO 14001).
- Diagramas UML e Arquitetura de Sistema: representação técnica do sistema para garantir clareza, modularidade e manutenção simplificada.
- Protótipo Interativo (Wireframe): simulação navegável das telas e funcionalidades, validando a experiência do usuário e a acessibilidade.
- Código-Fonte Versionado: cada módulo é armazenado em repositório Git, com controle de versões e histórico de alterações para rastreabilidade.
- Plano de Testes e Relatórios de Validação: garante que todas as funcionalidades estejam em conformidade com os requisitos de segurança e desempenho.
- Manual do Usuário e Documentação Técnica: assegura o uso correto do sistema e a continuidade das operações futuras.
- Relatório Final de Desempenho Ambiental e de Segurança: evidencia o cumprimento de práticas sustentáveis e normas de proteção de dados.

Controle de Qualidade

O controle de qualidade é contínuo e ocorre em todas as fases do desenvolvimento. Ele busca assegurar que os produtos entregues atendam aos padrões técnicos, regulatórios e de sustentabilidade definidos no escopo do projeto.

“Qualidade não é um ato, é um hábito. Manter a excelência requer processos contínuos de revisão, aprendizado e evolução.”

(Adaptado de ARISTÓTELES)

Práticas adotadas:

- Documentação Padronizada: utilização de templates normativos e versionamento de documentos para garantir integridade e auditoria.
- Rastreabilidade de Requisitos: uso de ferramentas como Jira e Confluence para mapear o ciclo de vida de cada requisito, desde a criação até a entrega final.
- Testes Automatizados: garantem agilidade, reduzindo falhas humanas e permitindo reavaliações rápidas a cada sprint.
- Auditorias Internas: realizadas ao fim de cada sprint para verificar conformidade com as normas da FDA 21 CFR Part 11 e GxP, bem como o impacto ambiental segundo ISO 14001.
- Revisões de Código e Validação Cruzada: promovem qualidade técnica, segurança da informação e alinhamento entre equipes multidisciplinares.

1.6 Continuidade do Projeto

Após a entrega final, o sistema passará por um plano de manutenção evolutiva e corretiva, com foco em:

- Atualização de normas regulatórias e patches de segurança.
- Monitoramento de desempenho e relatórios contínuos de sustentabilidade.
- Capacitação contínua de usuários e equipe técnica.
- Planejamento de novas sprints conforme feedback dos usuários e auditorias externas.

Essa continuidade garante longevidade, escalabilidade e conformidade permanente do sistema, mantendo-o atualizado e competitivo.

2. ALGORITMOS E ESTRUTURA DE DADOS EM PYTHON

2.1 Explicação geral do Código Fonte do Projeto

O projeto utiliza Python com estruturas de dados e aprendizado de máquina para realizar a estratificação de pacientes e a análise de remédios.

A Árvore AVL foi implementada para armazenar e organizar os dados dos remédios de forma balanceada, servindo como base para a rotina de aprendizado de máquina. Ela garante eficiência nas operações de inserção e busca, com complexidade $O(\log n)$.

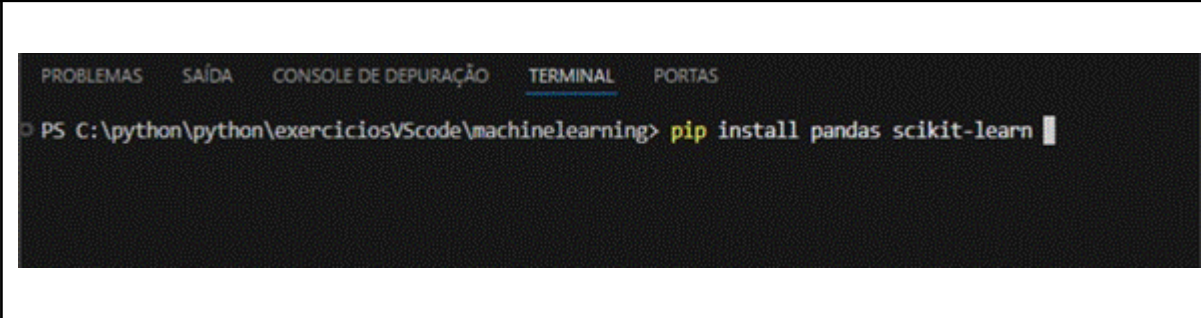
O Grafo é responsável por estruturar as informações dos pacientes e alimentar o modelo de aprendizado, que identifica e direciona automaticamente os pacientes prioritários com base na gravidade da doença. Ele também possui funções de fila para ordenar o atendimento dos mais graves.

O Main integra tudo: coleta os dados, executa o aprendizado de máquina (usando árvores de decisão do scikit-learn), realiza a detecção de outliers nos remédios e cria a fila de prioridade dos pacientes.

O código conta ainda com análise assintótica nas funções da AVL e do Grafo, e testes unitários, garantindo desempenho, precisão e confiabilidade.

O projeto foi separado em 4 arquivos dentro de uma única pasta, para poder reutilizar código, se necessário, e para uma melhor organização de tarefas.

Para poder rodar o código que vai ser mostrado logo em seguida, precisa das instalações das bibliotecas: pandas e scikit-learn, como mostrado abaixo.



```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
PS C:\python\python\exerciciosVScode\machinelearning> pip install pandas scikit-learn
```

Código do arquivo: [avltree.py](#)

```
from collections import deque

class No:
    def __init__(self, remedio=None, efetivo=None, colateral=None,
seguro=None, passa=None):
        # inicialização
        self.key = (passa, remedio)
        self.remedio = remedio
        self.passa = passa
        self.efetivo = efetivo
        self.colateral = colateral
        self.seguro = seguro
        self.height = 0
        self.parent = None
        self.left = None
        self.right = None

class AVLarvore:
    def __init__(self):
        self.root = None

    # Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
    recursão
    def insert(self, remedio, efetivo, colateral, seguro, passa):
        if self.root is None:
            self.root = No(remedio, efetivo, colateral, seguro,
passa)
        else:
            self._insert(remedio, efetivo, colateral, seguro, passa,
self.root)

    # Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
    recursão
    def _insert(self, remedio, efetivo, colateral, seguro, passa,
no_atual):
        new_key = (passa, remedio)
        current_key = (no_atual.passa, no_atual.remedio)

        if new_key < current_key:
            if no_atual.left is None:
```

```

        no_atual.left = No(remedio, efetivo, colateral,
seguro, passa)
        no_atual.left.parent = no_atual
        # Atualização altura do nó atual após inserção
        no_atual.height = 1 + max(
            self.get_height(no_atual.left),
            self.get_height(no_atual.right)
        )
        self._inspect_insertion(no_atual.left)
    else:
        self._insert(remedio, efetivo, colateral, seguro,
passa, no_atual.left)
        elif new_key > current_key:
            if no_atual.right is None:
                no_atual.right = No(remedio, efetivo, colateral,
seguro, passa)
                no_atual.right.parent = no_atual
                # Atualizar altura do nó atual após inserção
                no_atual.height = 1 + max(
                    self.get_height(no_atual.left),
                    self.get_height(no_atual.right)
                )
                self._inspect_insertion(no_atual.right)
            else:
                self._insert(remedio, efetivo, colateral, seguro,
passa, no_atual.right)
        else:
            print("Medicamentos já existem na arvore.")

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de
recursão
def print_tree(self):
    if self.root is not None:
        self._print_tree(self.root)

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de
recursão
def _print_tree(self, cur_node):
    if cur_node is not None:
        self._print_tree(cur_node.left)
        print(f'{cur_node.remedio}: passa={cur_node.passa},
efeito: {cur_node.efetivo}, colateral: {cur_node.colateral},

```

```

segurança: {cur_node.seguro} h={cur_node.height}')
    self._print_tree(cur_node.right)

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de
recursão
def height(self):
    if self.root is not None:
        return self._height(self.root, 0)
    else:
        return -1

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de
recursão
def _height(self, no_atual, altura_atual):
    if no_atual is None:
        return altura_atual - 1 # Ajuste para consistência

    left_height = self._height(no_atual.left, altura_atual + 1)
    right_height = self._height(no_atual.right, altura_atual + 1)
    return max(left_height, right_height)

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def find(self, passa, remedio):
    if self.root is not None:
        return self._find((passa, remedio), self.root)
    else:
        return None

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def _find(self, key, atual):
    key_atual = (atual.passa, atual.remedio)

    if key == key_atual:
        return atual # retorna o nó inteiro
    elif key < key_atual and atual.left is not None:
        return self._find(key, atual.left)
    elif key > key_atual and atual.right is not None:
        return self._find(key, atual.right)
    return None

```

```

# Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
recursão
def delete_value(self, passa, remedio):
    return self.delete_node(self.find(passa, remedio))

# Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
recursão
def delete_node(self, no):
    if no is None or self.find(no.passa, no.remedio) is None:
        print("No não achado na arvore.")
        return None

    def min_value_node(n):
        atual = n
        while atual.left is not None:
            atual = atual.left
        return atual

    def num_children(n):
        nc = 0
        if n.left is not None:
            nc += 1
        if n.right is not None:
            nc += 1
        return nc

    no_parent = no.parent
    no_children = num_children(no)

    if no_children == 0:
        if no_parent is not None:
            if no_parent.left == no:
                no_parent.left = None
            else:
                no_parent.right = None
        else:
            self.root = None

    elif no_children == 1:
        child = no.left if no.left is not None else no.right

        if no_parent is not None:

```

```

        if no_parent.left == no:
            no_parent.left = child
        else:
            no_parent.right = child
    else:
        self.root = child

    child.parent = no_parent

else: # no_children == 2
    sucessor = min_value_node(no.right)

    # copiar dados do sucessor para o nó
    no.key = sucessor.key
    no.passa = sucessor.passa
    no.remedio = sucessor.remedio
    no.efetivo = sucessor.efetivo
    no.colateral = sucessor.colateral
    no.seguro = sucessor.seguro

    # remover o sucessor
    self.delete_node(sucessor)
    return

if no_parent is not None:
    no_parent.height = 1 + max(
        self.get_height(no_parent.left),
        self.get_height(no_parent.right)
    )
    self._inspect_deletion(no_parent)

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def search(self, passa, remedio):
    if self.root is not None:
        return self._search((passa, remedio), self.root)
    else:
        return False

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def _search(self, key, no_atual):

```

```

    chave_atual = (no_atual.passa, no_atual.remedio)

    if key == chave_atual:
        return True
    elif key < chave_atual and no_atual.left is not None:
        return self._search(key, no_atual.left)
    elif key > chave_atual and no_atual.right is not None:
        return self._search(key, no_atual.right)
    return False

# Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(1)$ 
def _inspect_insertion(self, no_atual):
    if no_atual.parent is None:
        return

    left_height = self.get_height(no_atual.left)
    right_height = self.get_height(no_atual.right)

    if abs(left_height - right_height) > 1:
        path = [no_atual.parent, no_atual]
        # garantir que há x (taller child of no_atual)
        y = path[1]
        x = self.taller_child(y)
        self._rebalance_node(path[0], y, x)
        return

    new_height = 1 + max(self.get_height(no_atual.left),
self.get_height(no_atual.right))
    if new_height != no_atual.parent.height:
        no_atual.parent.height = new_height

    self._inspect_insertion(no_atual.parent)

# Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(1)$ 
def _inspect_deletion(self, no_atual):
    if no_atual is None:
        return

    left_height = self.get_height(no_atual.left)
    right_height = self.get_height(no_atual.right)

    if abs(left_height - right_height) > 1:

```

```

        y = self.taller_child(no_atual)
        x = self.taller_child(y)
        self._rebalance_node(no_atual, y, x)

    self._inspect_deletion(no_atual.parent)

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _rebalance_node(self, z, y, x):
    if y == z.left and x == y.left:
        self._right_rotate(z)
    elif y == z.left and x == y.right:
        self._left_rotate(y)
        self._right_rotate(z)
    elif y == z.right and x == y.right:
        self._left_rotate(z)
    elif y == z.right and x == y.left:
        self._right_rotate(y)
        self._left_rotate(z)
    else:
        raise Exception("Aconteceu um erro ao rebalancear.")

#Rotações da árvore para garantir uma árvore balanceada
# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _right_rotate(self, z):
    sub_root = z.parent
    y = z.left
    t3 = y.right
    y.right = z
    z.parent = y
    z.left = t3

    if t3 is not None:
        t3.parent = z
    y.parent = sub_root
    if y.parent is None:
        self.root = y
    else:
        if y.parent.left == z:
            y.parent.left = y
        else:
            y.parent.right = y

```

```

        z.height = 1 + max(
            self.get_height(z.left),
            self.get_height(z.right)
        )

        y.height = 1 + max(
            self.get_height(y.left),
            self.get_height(y.right)
        )

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _left_rotate(self, z):
    sub_root = z.parent
    y = z.right
    t2 = y.left
    y.left = z
    z.parent = y
    z.right = t2

    if t2 is not None:
        t2.parent = z
    y.parent = sub_root

    if y.parent is None:
        self.root = y
    else:
        if y.parent.left == z:
            y.parent.left = y
        else:
            y.parent.right = y

    z.height = 1 + max(
        self.get_height(z.left),
        self.get_height(z.right)
    )
    y.height = 1 + max(
        self.get_height(y.left),
        self.get_height(y.right)
    )

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def get_height(self, no_atual):

```

```

        if no_atual is None:
            return -1
        return no_atual.height

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def taller_child(self, no_atual):
    left = self.get_height(no_atual.left)
    right = self.get_height(no_atual.right)
    return no_atual.left if left >= right else no_atual.right

# Análise Assintótica Tempo: O(n) Espaço: O(n + h) - Lista +
Pilha recursão
def transforme_data(self, raiz, data_list):
    if raiz:
        self.transforme_data(raiz.left, data_list)
        data_list.append({
            "remedio": raiz.remedio,
            "efetivo": raiz.efetivo,
            "colateral": raiz.colateral,
            "seguro": raiz.seguro,
            "passa": raiz.passa
        })
        self.transforme_data(raiz.right, data_list)

# Análise Assintótica Tempo: O(n) Espaço: O(n) - Fila BFS
def bfs(self, raiz):

    if not raiz:
        return False

    fila = deque([raiz])
    passaram = []

    while fila:
        atual = fila.popleft()

        if atual.passa == 1:
            passaram.append(atual.remedio)

        if atual.left:
            fila.append(atual.left)

```

```

        if atual.right:
            fila.append(atual.right)

    return passaram

# Análise Assintótica Tempo: O(n) Espaço: O(n + h) - Lista +
Pilha recursão
def dfs(self, raiz, todos_remedios):

    if not raiz:
        return

    if raiz.remedio:
        todos_remedios.append(raiz.remedio)

    return self.dfs(raiz.left, todos_remedios) or
self.dfs(raiz.right, todos_remedios)

```

Código do arquivo: [grafo.py](#)

```

from collections import defaultdict
from collections import deque

class Grafo:
    # inicialização do grafo
    def __init__(self, vertices):
        self.vertices = vertices
        self.grafo = defaultdict(list)

        for nome, gravidade in self.vertices:
            self.grafo[nome].append(gravidade)

    #Análise Assintótica Tempo: O(n) Espaço: O(1)
    def todos_pacientes(self):
        for vizinhos in self.grafo:
            print(vizinhos)

    #Análise Assintótica Tempo: O(n) Espaço: O(n)

```

```

def alocar_fila_de_pioridade(self, pacientes_prioritarios):
    fila = deque([])
    for paciente in pacientes_prioritarios:
        fila.append(paciente)

    return fila

#Análise Assintótica Tempo: O(n) Espaço: O(n)
def mostrar_prioritarios(self, fila):

    for _ in range(len(fila)):
        print(f"Paciente: ", fila.popleft())

    #Análise Assintótica Tempo: O(n) Espaço: O(n)
def alocar_dados(self):
    data = []

    for key, value in self.grafo.items():
        data.append({"nome": key, "gravidade": value})

    return data

```

Código do arquivo: [main.py](#)

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn import tree
from avltree import AVLArvore
from grafo import Grafo

if __name__ == "__main__":
    # Criando a árvore para alimentar os dados da rotina de
    # aprendizado de máquina dos remédios.
    avl_tree = AVLArvore()
    dados_para_inserir = []
    vertices = []

    def menu():

        # Coletando dados para arvore AVL

```

```

        remedios_quantidade = int(input("Quantos remédios se vai
inserir para análise: "))

    for _ in range(remedios_quantidade):

print("\n-----\n")
        nome_remedio = input("Coloque o nome do remédio: ")

        eficiencia = input("Remédio é eficiente (s/n):
").lower().strip()

        if eficiencia not in ("s", "n"):
            print("Insira os dados corretamente.")
            return

        seguro = input("Remédio é seguro de administrar (s/n):
").lower().strip()

        if seguro not in ("s", "n"):
            print("Insira os dados corretamente.")
            return

        colateral = input("Colateral do remédio:
(sem/medio/alto): ").lower().strip()

        if colateral not in ("sem", "medio", "alto"):
            print("Insira os dados corretamente.")
            return

        if colateral in ("sem", "medio"):
            colateral_aceitavel = True
        else:
            colateral_aceitavel = False

        if colateral_aceitavel and eficiencia == "s" and seguro
== "s":
            passa = 1
        else:
            passa = 0

        dados_para_inserir.append((f'{nome_remedio}',

```

```

f'{eficiencia}', f'{colateral}', f'{seguro}', passa))

print("\n-----\n")

    # Coletando dados para o grafo que vai alimentar a rotina de
    # aprendizado de máquina dos pacientes.
    print("\n===== Coleta de dados dos pacientes
=====\\n")

    pacientes_quantidade = int(input("Quantos pacientes
participaram do teste: "))

    for _ in range(pacientes_quantidade):

        nome = input("Nome do paciente: ")
        gravidade_input = input("Doença é grave (s/n):
").lower().strip()
        if gravidade_input not in ("s", "n"):
            print("Insira os dados corretos.")
            return

        gravidade = 1 if gravidade_input == "s" else 0
        vertices.append((nome, gravidade))

    print("\n=====\\n")

    menu()

    # inserindo os dados na avl
    for remedio, efetivo, colateral, seguro, passa in
dados_para_inserir:
        avl_tree.insert(remedio, efetivo, colateral, seguro, passa)

    data_list = []
    # pegar os dados da AVL para fazer o aprendizado de máquina
    avl_tree.transforme_data(avl_tree.root, data_list)

    #dados que vão ser utilizados no aprendizado
    df = pd.DataFrame(data_list)

    inputs = df.drop('passa', axis='columns')
    target = df['passa']

```

```

#transformação de dados que estavam em string para números, pois
máquina apenas ler números
le_remedio = LabelEncoder()
le_efetivo = LabelEncoder()
le_colateral = LabelEncoder()
le_seguro = LabelEncoder()

inputs["remedio_n"] = le_remedio.fit_transform(inputs['remedio'])
inputs["efetivo_n"] =
le_colateral.fit_transform(inputs['efetivo'])
inputs["colateral_n"] =
le_colateral.fit_transform(inputs['colateral'])
inputs["seguro_n"] = le_seguro.fit_transform(inputs['seguro'])

inputs_n = inputs.drop(['remedio', 'efetivo', 'colateral',
'seguro'], axis="columns")
inputs_sem_numero = inputs.drop(['remedio_n', 'efetivo_n',
'colateral_n', 'seguro_n'], axis="columns")

print("\n ----- Dados -----")
print(inputs_sem_numero)
print("\n ----- Dados Codificados -----")
print(inputs_n)

model = tree.DecisionTreeClassifier()
model.fit(inputs_n, target)

outliers = []
#fazendo a análise dos dados, e mostrando o resultado na tela
print("\n===== Análise de remédios =====\n")
for i in range(len(inputs_n)):
    n1 = inputs_n.iloc[i]["remedio_n"]
    n2 = inputs_n.iloc[i]["efetivo_n"]
    n3 = inputs_n.iloc[i]["colateral_n"]
    n4 = inputs_n.iloc[i]["seguro_n"]

    nome_remedio = inputs.iloc[i]["remedio"]

    # Criando DataFrame para evitar warning
    treinamento_remedios = pd.DataFrame([[n1, n2, n3, n4]],
                                         columns=["remedio_n", "efetivo_n",
"colateral_n", "seguro_n"])

```

```

        resultado = model.predict(treinamento_remedios)

        print("=====")
        if resultado == 1:
            print(f"Remédio: {nome_remedio}. Aprovado para venda.")
        else:
            print(f"Remédio: {nome_remedio}. Reprovado, precisa de
mais testes e reajustes a serem feitos.")
            outliers.append((f"Nome do remédio: {nome_remedio}"))
        print("=====")

    #pegando os dados que passaram e todos que tiveram no teste.
    passaram = avl_tree.bfs(avl_tree.root)
    todos_remedios = []
    avl_tree.dfs(avl_tree.root, todos_remedios)

    print("\n===== Remédios que passaram no teste =====")
    for remedio in passaram:
        print(remedio)

    print("\n===== Todos remédios que estão no teste =====")
    for remedio in todos_remedios:
        print(remedio)

    print("\n===== Outliers detectados durante os testes =====")
    for outlier in outliers:
        print(outlier)
    print("\n")

    # Criando o grafo
    grafo = Grafo(vertices)
    data = grafo.alocar_dados()

    # Dados para a análise
    dfpacientes = pd.DataFrame(data)
    # utilizamos lambda para transformar o que era [1] ou [0] em 1 ou
0, assim conseguindo fazer o aprendizado de máquina
    dfpacientes['gravidade'] = dfpacientes['gravidade'].apply(lambda
x: x[0])

    inputs_pacientes = dfpacientes.drop('gravidade', axis="columns")
    target_pacientes = dfpacientes['gravidade']

```

```

#transformação de string para número
le_nome = LabelEncoder()
inputs_pacientes["nome_numero"] =
le_nome.fit_transform(inputs_pacientes['nome'])

inputs_n_pacientes = inputs_pacientes.drop(['nome'],
axis="columns")

modelPacientes = tree.DecisionTreeClassifier()
modelPacientes.fit(inputs_n_pacientes, target_pacientes)

print("\n===== Tabela dos pacientes =====")
print(inputs_pacientes)

pacientes_prioritarios = []

#fazendo a análise dos pacientes e o deslocamento para a fila de
prioridades.
print("\n===== Prevendo os pacientes =====")
for i in range(len(inputs_n_pacientes)):
    n = inputs_n_pacientes.iloc[i]["nome_numero"]
    nome = inputs_pacientes.iloc[i]["nome"]

    treinamento_paciente = pd.DataFrame([[n]],
columns=["nome_numero"])
    resultadopacientes =
modelPacientes.predict(treinamento_paciente)

    print(f"\nFazendo o deslocamento de pacientes para fila de
prioritários. Paciente: {nome}\n")
    if resultadopacientes == 1:
        pacientes_prioritarios.append(nome)

fila = grafo.alocar_fila_de_pioridade(pacientes_prioritarios)

print("\n===== Ordem de atendimento dos prioritários =====")
grafo.mostrar_prioritarios(fila)

```

Testes Unitários

Arquivo test_avltree.py:

```
import unittest
from avltree import AVLarvore
class TestAvl(unittest.TestCase):
    def setUp(self):
        print('setUp')
        self.dados = [('A', 's', 'sem colateral', 's', 1)]
        self.avl = AVLarvore()
        for remedio, efetivo, colateral, seguro, passa in self.dados:
            self.avl.insert(remedio, efetivo, colateral, seguro,
passa)

    def test_bfs(self):
        # Com apenas 1 elemento, deve retornar apenas ['A']
        self.assertEqual(self.avl.bfs(self.avl.root), ['A'])

    def test_dfs(self):
        todos_remedios = []
        self.avl.dfs(self.avl.root, todos_remedios)
        # Com apenas 1 elemento, deve retornar apenas ['A']
        self.assertEqual(todos_remedios, ['A'])

    def test_transforme_data(self):
        data_list = []
        self.avl.transforme_data(self.avl.root, data_list)
        self.assertEqual(data_list, [{'remedio': 'A', 'efetivo': 's',
'colateral': 'sem colateral', 'seguro': 's', 'passa': 1}])

    def test_search(self):
        self.assertEqual(self.avl.search(1, 'A'), True)

    def test_height(self):
        self.assertEqual(self.avl.height(), 0)

if __name__ == '__main__':
    unittest.main()
```

Arquivo test_grafo.py:

```
import unittest
from grafo import Grafo
from avltree import AVLarvore
import io
import sys
from collections import deque

class TestGrafo(unittest.TestCase):
    def setUp(self):
        print('setUp')
        self.vertices = [("A", 1), ("B", 0), ("C", 1)]
        self.grafo = Grafo(self.vertices)

    def test_todos_pacientes(self):
        captured = io.StringIO()
        sys.stdout = captured
        self.grafo.todos_pacientes()
        sys.stdout = sys.__stdout__

        saida = captured.getvalue().strip().split("\n")
        self.assertEqual(saida, ["A", "B", "C"])

    def test_alocar_fila_de_pioridade(self):
        pacientes_prioritarios = ["A", "B", "C"]

self.assertEqual(self.grafo.alocar_fila_de_pioridade(pacientes_priori
tarios), deque(['A', 'B', 'C']))

    def test_mostrar_prioritarios(self):
        fila = deque(['A', 'B', 'C'])
        captured = io.StringIO()
        sys.stdout = captured
        self.grafo.mostrar_prioritarios(fila)
        sys.stdout = sys.__stdout__

        saida = captured.getvalue().strip().split("\n")
        self.assertEqual(saida, [
            "Paciente número 1: A",
            "Paciente número 2: B",
```

```
        "Paciente número 3: C"
    ])

    def test_alocar_dados(self):

        self.assertEqual(self.grafo.alocar_dados(), [{ 'nome': 'A',
'gravidade': [1]}, { 'nome': 'B', 'gravidade': [0]}, { 'nome': 'C',
'gravidade': [1]}])

if __name__ == '__main__':
    unittest.main()
```

3. EDUCAÇÃO AMBIENTAL

3.1 Introdução

O avanço tecnológico e científico na área da saúde tem proporcionado um cenário em que a integração entre pesquisa clínica e produção farmacêutica se torna essencial para o desenvolvimento de novos medicamentos e tratamentos mais eficazes. No entanto, a gestão dessas informações ainda enfrenta desafios, principalmente no que diz respeito à segurança dos dados, controle de processos e sustentabilidade ambiental.

Dessa forma, o presente trabalho tem como objetivo propor o desenvolvimento de uma plataforma integrada que permita otimizar os processos de pesquisa clínica e produção farmacêutica, contemplando também ações voltadas à educação ambiental, como o descarte correto de medicamentos vencidos ou inutilizados.

3.2 Justificativa

O descarte inadequado de medicamentos representa um problema crescente de saúde pública e ambiental. Substâncias químicas provenientes desses produtos, quando descartadas em lixo comum ou rede de esgoto, contaminam o solo e a água, afetando ecossistemas e colocando em risco a saúde humana.

Diante disso, o desenvolvimento de uma plataforma que, além de gerenciar a pesquisa clínica e a produção farmacêutica, também eduque usuários sobre práticas seguras e sustentáveis, torna-se uma contribuição relevante tanto para o meio acadêmico quanto para a sociedade.

Além disso, o uso de linguagens de programação, como Python, facilita a criação de sistemas interativos que podem auxiliar nessa conscientização ambiental de forma acessível e eficiente.

“O descarte inadequado de medicamentos no lixo doméstico ou no esgoto representa risco significativo ao meio ambiente e à saúde pública, exigindo políticas e ações educativas permanentes.”

(BRASIL. Ministério do Meio Ambiente, 2010, p. 22).

3.3 Objetivos

Objetivo Geral

Desenvolver uma plataforma digital integrada voltada à gestão da pesquisa clínica e produção farmacêutica, incorporando funcionalidades educativas relacionadas ao descarte correto de medicamentos.

Objetivos Específicos

- Automatizar processos de registro, controle e acompanhamento de pesquisas clínicas e lotes farmacêuticos.
- Criar ferramentas interativas que promovam a conscientização ambiental.
- Aplicar conceitos de segurança da informação no armazenamento de dados clínicos.
- Incentivar o consumo e descarte responsável de medicamentos.

3.4 Fundamentação Teórica

A transformação digital na indústria farmacêutica tem se mostrado essencial para o avanço das pesquisas clínicas. Plataformas integradas permitem o armazenamento centralizado de dados, a otimização de fluxos de trabalho e a redução de erros humanos, como destaca Pressman (2016) ao tratar da importância de sistemas bem estruturados no ciclo de vida do software.

Paralelamente, a educação ambiental surge como instrumento fundamental de conscientização e mudança de comportamento. Segundo a Política Nacional de Educação Ambiental (Lei nº 9.795/1999), é dever de todos promover a educação voltada à preservação do meio ambiente, o que inclui o descarte correto de medicamentos.

Assim, unir tecnologia e responsabilidade ambiental representa uma abordagem inovadora e necessária, que fortalece os princípios da sustentabilidade e da saúde pública.

3.5 Metodologia

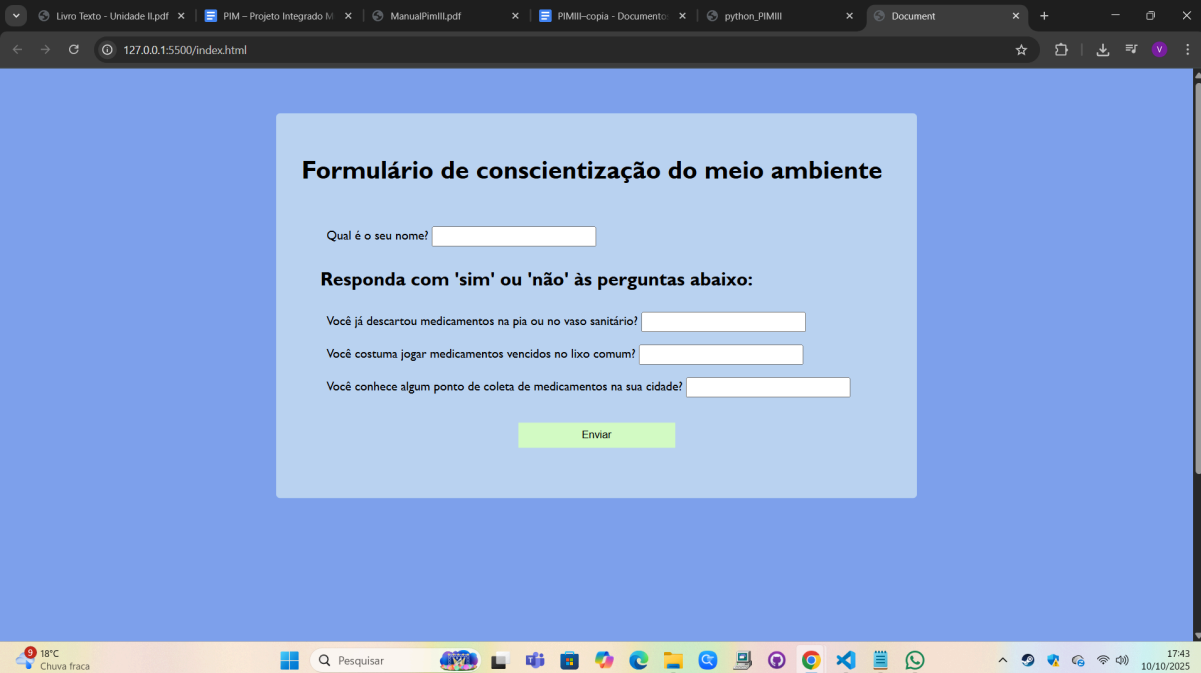
O projeto foi estruturado em quatro etapas principais:

- Análise e levantamento de requisitos: identificação das necessidades relacionadas à gestão de pesquisa e produção farmacêutica.
- Desenvolvimento do sistema: construção de um protótipo interativo em Python, com interface textual simples e intuitiva.
- Integração de boas práticas ambientais: criação de um módulo educativo sobre descarte consciente.
- Testes e validações: verificação do funcionamento e clareza das informações apresentadas aos usuários.

3.6. Atividade de extensão

Para atividade de extensão, resolvemos desenvolver uma website de conscientização ambiental, na qual o sistema faz perguntas ao usuário sobre o descarte correto de medicamentos. Ao final, ele fornece orientações de acordo com as respostas dadas, desenvolvido com JavaScript, HTML e CSS.

Antes e após de fazer a coleta de dados:



The screenshot shows a web browser window with several tabs open. The active tab displays a form titled "Formulário de conscientização do meio ambiente". The form is set against a light blue background. It includes a text input field for the user's name, followed by a prompt to answer questions with 'sim' or 'não'. There are three more text input fields for specific questions about medication disposal. A green "Enviar" button is at the bottom of the form. The browser's address bar shows the URL "127.0.0.1:5500/index.html". The Windows taskbar at the bottom shows the time as 17:43 on 10/10/2025.

Formulário de conscientização do meio ambiente

Qual é o seu nome?

Responda com 'sim' ou 'não' às perguntas abaixo:

Você já descartou medicamentos na pia ou no vaso sanitário?

Você costuma jogar medicamentos vencidos no lixo comum?

Você conhece algum ponto de coleta de medicamentos na sua cidade?

Formulário de conscientização do meio ambiente

Qual é o seu nome?

Responda com 'sim' ou 'não' às perguntas abaixo:

Você já descartou medicamentos na pia ou no vaso sanitário?

Você costuma jogar medicamentos vencidos no lixo comum?

Você conhece algum ponto de coleta de medicamentos na sua cidade?

--- Resultados e Orientações ---

- Evite descartar medicamentos na água! Isso contamina rios e solos, prejudicando o meio ambiente.
- Medicamentos no lixo comum podem causar contaminação. Procure um ponto de coleta autorizado.
- Muito bem! Continue utilizando os pontos de coleta de forma correta.

Obrigado por participar, Vitória! Sua atitude ajuda a proteger o meio ambiente e a saúde pública.

3.7 Resultados Esperados

Com a implementação da plataforma, espera-se alcançar:

- Maior eficiência na gestão de dados de pesquisa clínica e produção.
- Engajamento dos usuários em práticas sustentáveis.
- Redução do impacto ambiental causado pelo descarte incorreto de medicamentos.
- Valorização da responsabilidade social dentro das instituições de saúde e farmacêuticas.

“A tecnologia deve ser uma aliada da sustentabilidade, transformando informação em consciência e consciência em ação.”

(Adaptado de CAPRA, Fritjof. *As conexões ocultas: ciência para uma vida sustentável*. São Paulo: Cultrix, 2002.)

Conclusão

Com o desenvolvimento deste projeto, foi possível perceber como a **tecnologia pode ser uma grande aliada da saúde e do meio ambiente**. A criação de uma plataforma que integra gestão farmacêutica, pesquisa clínica e educação ambiental mostra que é possível unir inovação, segurança e sustentabilidade em um mesmo sistema.

Além de facilitar o controle de dados e processos, a proposta também incentiva o **descarte consciente de medicamentos**, contribuindo para a preservação do planeta e o bem-estar das pessoas.

Por fim, este trabalho reforça a importância de **continuar investindo em soluções tecnológicas responsáveis**, que respeitem as normas, protejam as informações e ajudem na construção de um futuro mais sustentável e humano.

Bibliografia

LEANDRO. O que é UML e Diagramas de Caso de Uso: Introdução Prática à UML. DEVMEDIA. 2012. Disponível em:

<https://www.devmedia.com.br/o-que-e-uml-e-diagramas-de-caso-de-uso-introducao-pratica-a-uml/23408>. Acesso: 03 de setembro

FÁBIO. Desmistificando o Product Owner. DEVMEDIA. 2013. Disponível em:

<https://www.google.com/amp/s/www.devmedia.com.br/amp/desmistificando-o-product-owner/27793> - Acesso: 03 de setembro

SCHWABER, Ken. A Little History About Us. [Scrum.org](https://www.scrum.org). Disponível em:

<https://www.scrum.org/about> - Acesso: 03 de setembro

CAMARGO, Robson. Backlog: aprenda o que é e como compatibilizar com o fluxo de trabalho. robsoncamargo. 04 de dezembro de 2019. Disponível em:

<https://robsoncamargo.com.br/blog/Backlog-o-que-e-e-como-pode-ajudar-em-seus-projetos> - Acesso: 03 de setembro

Scrum, a arte de fazer o dobro na metade do tempo. Cromo. 24 de junho de 2020. Disponível em:

https://cromoconsultoria.com.br/scrum-a-arte-de-fazer-o-dobro-na-metade-do-tempo/?gad_source=1&gad_campaignid=16608158376&gbraid=0AAAAABdi1dwzBHCAu_mzWGdXLB0O_gsL3&gclid=Cj0KCQjw0Y3HBhCxARIsAN7931Ukow1rGz-Mzc9U34n1Vggbp_EEadyrKETH539iK_k_hNcxU2Jlq7EaAsuKEALw_wcB - Acesso: 03 de setembro

EGRAD_ARQ - ESTUDO CONTEMPORÂNEO E TRANSVERSAL: GESTÃO AO ÁGIL DE PROJETOS - 2025 - 51 - 1. apiron. Disponível em:

<https://www.modelitos.com.br/atividades/questionario/questionario-estudo-contemporaneo-e-transversal-gestao-agil-de-projetos-av-20255> - Acesso: 03 de setembro

RADIGAN, Dan. Backlog do produto: dicas para criação e priorização.

ATLASSIAN. Disponível em:

<https://www.atlassian.com/br/agile/scrum/backlogs> - Acesso: 03 de setembro

<https://qualificabr.com/fda-21-cfr-part-11-saiba-mais-sobre-essa-norma/> -

Acesso: 03 de Outubro

REHKOPF, Max. O que são sprints na gestão de projetos. ATLASSIAN.

Disponível em:

<https://www.atlassian.com/br/agile/scrum/sprints> . Acesso: 03 de Outubro
Certificações ISO: o que são, como funcionam e os benefícios. CATALISA.
Disponível em:

https://catalisajr.com.br/certificacoes-iso/?utm_source=google&utm_medium=blog&utm_campaign=br-midia_paga-texto_de_blog-google-visitas_blog-certificacoes_iso_certificacoes_iso&gad_source=1&gad_campaignid=1718670367&gbraid=0AAAAADK059ag0FADGQr5aodoCMXyc4WWz&gclid=CjwKCAjw6P3GBhBVEiwAJPjmLi0vSH_957heWcTITEz-oOkBIIDQats4pNUibaWo0U2Z2I7QXZqufRoCQgkQAvD_BwE. Acesso: 03 de Outubro

PESSÕA, Camilia. Padrões arquiteturais: arquitetura de software descomplicada. alura. Disponível em:

<https://www.alura.com.br/artigos/padroes-arquiteturais-arquitetura-software-descomplicada?srsItd=AfmBOooQUHYJE3NzclxlQQlyUljNtsxpS68XjR2QOPHFIWafCJpWCR0I>. Acesso: 03 de Outubro

ALMEIDA, Washington. Diferença entre Garantia de Qualidade e Controle de Qualidade em Engenharia de Software. 31 de Outubro de 2024. Disponível em:

<https://blog.grancursosonline.com.br/diferenca-entre-garantia-de-qualidade-e-controle-de-qualidade-em-engenharia-de-software/amp>. Acesso: 06 de outubro

Entregáveis de projeto: o que são e como definir?. FlowUp. Disponível em:

<https://www.flowup.me/blog/entregaveis-de-projeto/> - Acesso: 06 de outubro

Controle de entregáveis: como evitar atrasos e aumentar a eficiência da sua obra. coordly. 14 de março de 2025. Disponível em:

<https://coordly.co/blog/controle-de-entregaveis-como-evitar-atrasos-e-aumentar-a-eficiencia-da-sua-obra/> . Acesso: 06 de outubro

GIL ESPINHA, Roberto. Entenda o que são os entregáveis de um projeto e como definir. artia. 17 de abril de 2023. Disponível em:

<https://artia.com/blog/entregaveis-de-um-projeto/>. Acesso: 06 de Outubro

Machine Learning Tutorial Python - 9 Decision Tree. codebasics. 16 de novembro de 2018. Disponível em:

<https://www.youtube.com/watch?v=PHxYNGo8Ncl&t=366s> Acesso: 04 de setembro de 2025.

AVL Tree: Background & Python Code. Bryan Faure. 21 de fevereiro de 2018. Disponível em:

<https://www.youtube.com/watch?v=lxHF-mVdwK8&t=1s..> Acesso: 04 de setembro de 2025.

BRASIL. Lei nº 9.795, de 27 de abril de 1999. Dispõe sobre a educação ambiental e institui a Política Nacional de Educação Ambiental. Diário Oficial da União, Brasília, DF, 1999. Acesso: 7 de Outubro.

SILVA, J. P.; OLIVEIRA, R. M. Gestão de Resíduos de Medicamentos: desafios e perspectivas. Revista Saúde & Ambiente, v. 25, n. 3, p. 45-58, 2022. Acesso: 7 de Outubro.

UNIP. Manual do Projeto Integrado Multidisciplinar (PIM). São Paulo: Universidade Paulista, 2025. Acesso: 7 de Outubro.

AGÊNCIA NACIONAL DE VIGILÂNCIA SANITÁRIA (ANVISA). Guia para o descarte de medicamentos. Brasília: ANVISA, 2023. Disponível em: <https://www.gov.br/anvisa>. Acesso: 7 de Outubro.